| | School of Engineering & Technology | |
|---|---|---|
| | **Department: SOET** | **Session: 2025-26** |
| | **Program: BTech** | **Semester: IV** |
| | **Course Code: ENCS256** | **Number of students: 715** |
| | **Course Name: Analysis and Design of Algorithms Lab** | **Faculty: Dr. Aarti** |

## Lab Assignment -1: Algorithm Foundations Instructions:

- Assignment must be submitted within the deadline communicated by the instructor at the time of release.
- Assignment must be submitted on https://lms.krmangalam.edu.in/
- You must provide a link to your GitHub repository with your submission on **LMS**.
- Use of **ChatGPT** and similar tools is strictly prohibited.
- The assignment needs to be submitted by each individual.
- This assignment carries a total of **10 marks**.
- Assignment will be assessed based on the evaluation rubrics.
- Estimated Duration: 5-6 hours

**Assignment Title**: **Understanding Algorithm Efficiency through Analysis, Recurrences, and Sorting**

---

### Real-World Problem Context

In real-world software systems, efficiency is as important as correctness. Whether designing a search engine, database engine, or embedded system, developers must evaluate algorithms based on their **time and space behaviour** rather than intuition alone. This assignment introduces foundational concepts such as **algorithm analysis, asymptotic notations, recurrence relations, and sorting techniques**, enabling students to reason about scalability and performance before implementation.

---

### Learning Objectives

By completing this project, you will:

1. Understand the role and importance of algorithms in problem-solving.
2. Analyze algorithms using **time and space complexity** models.
3. Apply **asymptotic notations** to compare algorithm efficiency.
4. Solve **recurrence relations** using standard techniques.

5. Interpret the **rate of growth** of algorithms for large inputs.

---

**Assignment Tasks**

**Task 1: Algorithm Growth Observation**

**Write Python programs to implement the following functions:**

1. Constant Time : $O(1)$
2. Linear Time : $O(n)$
3. Quadratic Time : $O(n^2)$
4. Logarithmic Time : $O(\log n)$

**Steps**

- Measure execution time for different input sizes (e.g., n = 10, 100, 500, 1000).
- Plot Input Size vs Execution Time.
- Observe and comment on the growth pattern.

**Deliverables**

- Python script or notebook
- Execution time plots

**Task 2: Best, Average, and Worst Case Analysis**

Implement **Linear Search** and **Binary Search**.

**Requirements**

- Generate random arrays of different sizes.
- Measure execution time for:
  - Best case
  - Average case
  - Worst case
- Compare results using plots.

**Deliverables**

- Search algorithm implementations
- Comparison plots
- Observations in comments/markdown

**Task 3: Recursion and Recurrence Validation**

Implement the following recursive algorithms:

1. Factorial
2. Fibonacci (Naïve Recursive)
3. Fibonacci (Dynamic Programming)

**Steps**

- Count number of function calls for each input size.
- Measure execution time.
- Compare recursive vs optimized approach.

**Deliverables**

- Python implementations
- Table comparing execution time and function calls

**Task 4: Solving Recurrences through Code**

Implement recursive functions corresponding to these recurrences:

1. $T(n) = T(n/2) + n$
2. $T(n) = 2T(n/2) + n$

**Steps**

- Count recursive calls.
- Measure execution time.
- Validate results with expected asymptotic complexity.

**Deliverables**

- Code with call counters
- Observational comments

---

**Evaluation Rubric (Total: 10 Marks)**

| Criteria | Marks | Description |
|---|---|---|
| 1. Correct Algorithm Implementation | **3** | Accurate and working code |
| 2. Performance Measurement | **2** | Proper timing and input scaling |
| 3. Graphs & Visualization | **2** | Clear, labeled, meaningful plots |
| 4. Observations & Analysis | **2** | Logical interpretation of results |
| 5. Code Quality & Structure | **1** | Readability, comments, modularity |

---

**Submission Instructions**

Your GitHub repository must contain:

- README.md – overview & execution instructions
- algorithms_assignment1.py / analysis.ipynb / algorithms_assignment1.cpp
- plots/ – saved graphs
- requirements.txt

Submit **GitHub repository** link on LMS.

---

**Contact for Queries**: Dr. Aarti, aarti.sangwan@krmangalam.edu.in